Detecting Machine-Generated Code in Multi-Lingual, Multi-Generator and Multi-Domain Settings

A Hybrid Approach: CodeT5 + AST Feature Fusion

Dataset: CoDET-M4 | Binary Classification | Python · Java · C++ | 500,552 Samples

Shivi Goel · Anika Aharwal · Ashwin Verma

NLP End-Semester Project Report

PROBLEM STATEMENT

Why Is This Hard?

LLMs like GPT-4, Claude, and Gemini produce

- syntactically correct,
- functionally coherent code

across multiple languages, making it nearly impossible to tell apart from human-written code.

This creates critical challenges in:

Academic Integrity

Unfair advantage in coding assignments

Software Security

Hidden vulnerabilities in AI code

IP Attribution

Ownership disputes in codebases

Code Review

Auditing AI-contributed code at scale

Why Simple Approaches Fail

- AI code follows valid syntax & semantics: surface analysis is insufficient
- Human styles vary enormously by experience, domain & preference
- AI models produce diverse outputs based on architecture & prompts
- Multilingual code: Python, Java, C++ have distinct idioms
- Pattern-matching fails: AI mirrors human conventions closely
- Deep semantic + structural analysis required

DATASET

CoDET-M4 Dataset

500,552
Total Samples

3
Languages

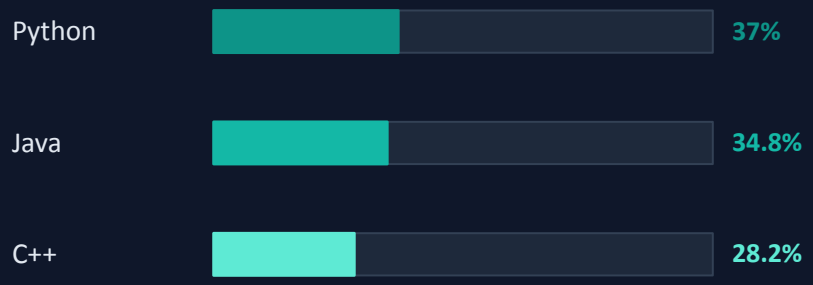
4+
AI Models

~50/50
AI vs Human

Dataset Splits

Split	Samples	AI	Human
Train	405,069	207,982	197,087
Validation	47,734	~23,900	~23,800
Test	47,749	~23,850	~23,900
Total	**500,552**	**254,331**	**246,221**

Language Distribution (Train)



AI Models Included:

GPT-4 · Claude · Gemini · Mixtral

DATA PROCESSING

Preprocessing Pipeline

1 Null Removal

Drop samples missing code, language, target or split fields

2 Deduplication

Remove exact duplicates on cleaned_code to prevent leakage

3 Length Filter

Remove snippets < 20 chars (insufficient structure)

4 Label Encoding

human → 0
ai → 1 (binary)

Post-Preprocessing Effective Dataset Sizes

395,805	46,956	47,132	~50/50
Training samples	Validation samples	Test samples	Label Balance AI / Human

⚡ *Deep model training used stratified subsampling: 3,000 AI + 3,000 human per language = 18,000 samples (4.5% of full training set)*

BASELINE 1

SVM + TF-IDF

Architecture

Feature Extraction

Character-level TF-IDF vectorizer
N-grams: length 3–5 characters
Min doc freq: 2 | Max features: 50,000

Why Char N-grams?

Captures sub-token patterns:
— Indentation styles
— Naming conventions
— Operator spacing

Classifier

LinearSVC (efficient with
high-dimensional sparse vectors)
C tuned via grid search: {0.01, 0.1, 1, 5, 10}

Macro F1: 0.9534 | Trained on full 395,805 samples

Test Set Results (47,132 samples)

Metric	Human	AI
Precision	0.954	0.957
Recall	0.960	0.950
F1-Score	0.955	0.960
Accuracy	—	**95.55%**

Key Takeaway

Surprisingly strong for a classical ML approach. Character n-gram statistics carry substantial signal — confirms AI code has distinctive surface patterns. Best C = 1.

BASELINE 2

CodeBERT Fine-tuning

Architecture & Training

Model

microsoft/codebert-base
125M parameters | Encoder-only

Data

Train: (18,000, 7)
Validation: (46,355, 7)
Test: (46,459, 7)

Pre-training

CodeSearchNet corpus
Masked LM + Replaced Token Detection
Bimodal: code + NL documentation pairs

Classification Head

[CLS] token (768-dim) ⟶ Linear ⟶ 2 classes
Inputs prefixed: 'language: python code: ...'
Max sequence length: 256 tokens

Training Config

AdamW lr=2e-5, weight decay=0.01
Cosine LR schedule + warmup (6%)
Early stopping (patience=2), max 3 epochs

Test Set Results (approx.)

Metric	Human	AI
Precision	~0.970	~0.960
Recall	~0.961	~0.970
F1-Score	~0.970	~0.970
Accuracy	—	**~97.1%**

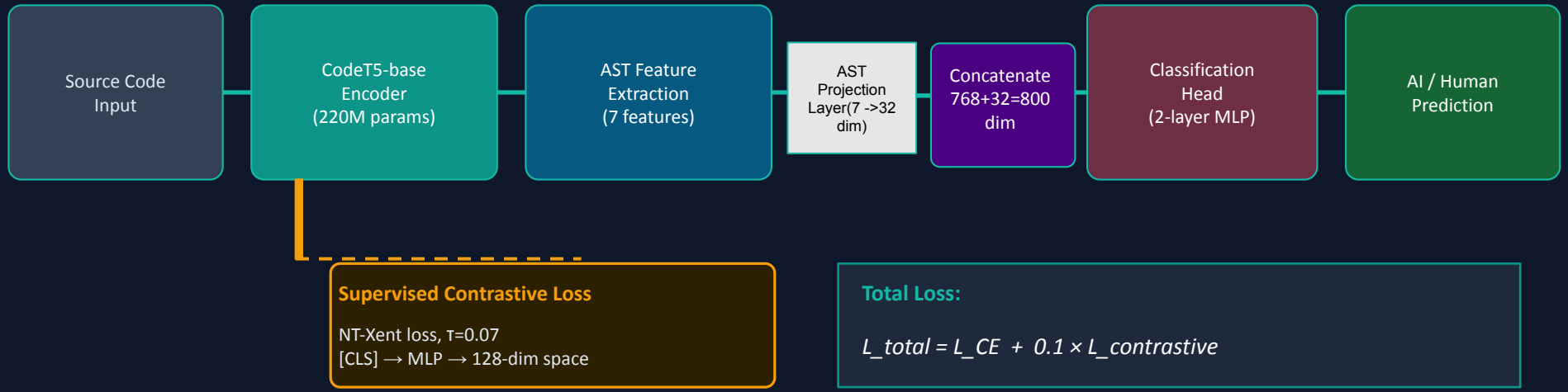
Why Better Than SVM?

Bidirectional attention captures long-range token dependencies invisible to bag-of-n-grams. Pre-training on code+NL enables deep semantic understanding beyond surface statistics.

Macro F1: ~0.971 | Converged in 3 epochs (early stopping)

PROPOSED MODEL

CodeT5 + AST Fusion Architecture



Key Design Choices

CodeT5 over CodeBERT: CodeT5 identifier-aware denoising learns richer structural representations

Late Fusion: Concatenate after independent feature extraction to preserve both signal types

AST Features: Explicit structural info: loops, conditionals, assignments — complementary to semantics

Contrastive Loss: Pushes AI vs Human embeddings apart ⟶ more discriminative representation space

PROPOSED MODEL

AST Feature Extraction

7 Python AST Features (via ast module)

`ast.FunctionDef`	Function definitions
`ast.AsyncFunctionDef`	Async function definitions
`ast.ClassDef`	Class definitions
`ast.For + ast.While`	Loop constructs (combined)
`ast.If`	Conditional statements
`ast.Assign`	Assignment statements
`ast.Import + ast.ImportFrom`	Import statements (combined)

Java & C++ Handling

Python's ast module cannot parse Java or C++. For these languages, a zero vector of dimension 7 is used as the AST feature representation. Normalization (zero mean, unit variance) applied before fusion.

Why AST Features Help

- AI code: more balanced AST depth distributions
- More uniform use of control structures
- Fewer irregular or deeply nested constructs
- Explicit structural signal complementary to semantic embeddings
- Language-agnostic structural representation

PROPOSED MODEL — RESULTS

Training Dynamics

Epoch-by-Epoch Metrics

Epoch	Train Loss	Train F1	Val Loss	Val F1	Note
1	0.3072	0.9340	0.1731	**0.9577** ★	New Best
2	0.1472	0.9734	0.1231	**0.9659** ★	New Best
3	0.0993	0.9823	0.0964	**0.9750** ★	New Best
4	0.0677	0.9904	0.1180	0.9694	
5	0.0539	0.9925	0.0966	0.9749	Early Stop

Best model checkpoint: Epoch 3 (Val F1 = 0.9750)

Training Config

- Optimizer: AdamW
- LR: 2e-5
- Weight decay: 0.01
- LR: Cosine + warmup
- Batch: 32 (×2 accum)
- Max epochs: 7
- Early stop: patience 2
- Subsample: 18,000

Data

Train: (18,000, 7)
Validation: (46,355, 7)
Test: (46,459, 7)

PROPOSED MODEL — RESULTS

CodeT5 + AST Fusion: Test Results

97.68%	0.9697	0.9827	0.9762
Accuracy	Precision (macro)	Recall (macro)	F1-Score

Per-Class Results (46,459 test samples)

Class	Precision	Recall	F1-Score
Human (0)	0.9800	0.9700	0.9750
AI (1)	0.9697	0.9827	0.9800

Error Pattern Insight

More false positives (690) than false negatives (390): model is slightly more prone to labeling human code as AI-generated. In high-stakes contexts (e.g. academic screening), this is the safer failure mode — better to over-flag than to miss AI code.

Confusion Matrix

23,262 True Negative	**690** False Positive
390 False Negative	**22,117** True Positive

Predicted →

COMPARISON

Model Comparison

Model	Accuracy	Precision	Recall	Macro F1	Train Size
SVM + TF-IDF	95.55%	0.9573	0.9496	0.9534	395,805
CodeBERT	~97.1%	~0.962	~0.979	~0.971	full train
CodeT5 + AST ★	**97.68%**	**0.9697**	**0.9827**	**0.9762**	**18,000 (4.5%)**

Key Findings

Data Efficiency: CodeT5+AST beats both baselines using only 4.5% of training data

Recall Win: AI recall 0.9827 vs 0.9496 (SVM) — 6pp fewer missed AI samples

AST Value: Structural features complement semantics from pre-trained encoder

Contrastive Learning: Better embedding separation improves discrimination

CONCLUSION

Prompts & Contributions

Prompts

Explain the structure of the CoDet-M4 dataset and its labels for machine-generated vs human-written code detection.

Which features in source code help distinguish machine-generated code from human-written code?

How to choose the best n-gram range for character-level TF-IDF in code classification?

How to fine-tune CodeBERT for binary classification of machine-generated vs human-written code?

Extract structural features from Python code using Abstract Syntax Trees.

Member Contributions

Anika Aharwal

Data preprocessing, CodeT5 research, report, presentation

Shivi Goel

SVM + TF-IDF (baseline 1) implementation, CodeT5 research, report

Ashwin

CodeBERT (baseline 2) implementation, CodeT5 implementation